Complete Web App / SaaS Deployment, Security & Maintenance Prompt Pack

Based on the handwritten requirements provided • Designed for AI coding agents, vibe-coding tools, and manual development workflows

Purpose: Turn the notes into a repeatable production workflow covering development, deployment, domains, authentication, databases, analytics, security, SEO, code review, monitoring, subscriptions, storage management, international compliance, and web-to-mobile conversion.

Layer	Recommended responsibility	Examples
Source control	Versioning, branches, CI/CD	GitHub
Frontend hosting	Static/SSR web app	Vercel, Netlify, Cloudflare Pages
Backend	API/server/jobs	Railway, Render, Fly.io, Cloud Run
Database/Auth/Storage	Users, data, files	Supabase / Firebase
DNS/CDN/Security	DNS, TLS, WAF, caching	Cloudflare
Domain	Domain registration	Cloudflare Registrar, Namecheap, GoDaddy, Hostinger
Monitoring	Errors, uptime, performance	Sentry, provider logs, uptime monitor
Analytics	Product usage and funnels	Google Analytics / privacy-conscious analytics
Email	Transactional/auth emails	Resend, Postmark, SendGrid, provider SMTP

0. Master Production Prompt

```
You are a senior full-stack engineer, DevOps engineer, security engineer, product engineer and technical auditor.

Take the existing web app exactly as it currently exists. Do not blindly rewrite it. First inspect the repository, package files, environment variables, routes, frontend, backend/API, database schema, authentication, storage, deployment configuration, domain/DNS configuration, third-party integrations and build scripts.

Goal: turn the app into a production-ready, secure, maintainable product.

Work in this order:
1. Audit the existing architecture and identify what is already implemented.
2. Create a deployment architecture appropriate for the stack.
3. Separate frontend, backend/API, database, storage and secrets.
4. Configure secure authentication using email/password and Google OAuth where appropriate.
5. Configure database, storage quotas and cleanup policies.
6. Configure domain, DNS, HTTPS and Cloudflare where useful.
7. Configure analytics, error monitoring, uptime monitoring and product event tracking without collecting unnecessary sensitive data.
8. Configure transactional email.
9. Review every API endpoint for authentication, authorization, validation, rate limiting and abuse protection.
10. Move all secrets to environment variables and ensure they are never committed or exposed to the browser.
11. Review the codebase with static analysis and AI-assisted code review, then manually verify important findings.
12. Configure SEO, metadata, sitemap, robots.txt, canonical URLs and structured data where relevant.
13. Add subscription/usage-limit logic if the product is paid.
14. Add production logging, backups, rollback strategy and incident-response basics.
15. Test the entire application before declaring the work complete.

Important rules:
- Never expose server-side secrets in client-side code.
```

```
- Never put production secrets in Git.
- Never delete user data automatically without a documented retention policy and a safe recovery/backup strategy.
- Do not claim a deployment is successful until build, runtime, database, authentication, domain, email and critical user flows have been tested.
- Preserve existing features unless a change is required for security, correctness or deployment.
- Produce a final report listing files changed, environment variables required, deployment steps, security findings, unresolved risks and test results.
```

1. Development Approach — Manual, AI-Assisted or Vibe Coding

Objective: Review the existing project and determine which parts should be written manually and which can safely be accelerated with AI/vibe coding. Do not generate large amounts of unreviewed code. For every AI-generated change, explain its purpose, dependencies, security impact and test coverage. Keep business logic understandable and maintainable.

```
Act as a senior production engineer working on this specific area: 1. Development Approach — Manual, AI-Assisted or Vibe Coding.

TASK
Review the existing project and determine which parts should be written manually and which can safely be accelerated with AI/vibe coding. Do not generate large amounts of unreviewed code. For every AI-generated change, explain its purpose, dependencies, security impact and test coverage. Keep business logic understandable and maintainable.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

2. GitHub Repository & Source-Control Setup

Objective: Configure the project as a professional Git repository. Create a clean README, sensible folder structure, branch strategy, .gitignore, environment-variable documentation, issue templates if useful, and CI checks. Ensure secrets, local databases, build artifacts, logs, private keys and service credentials are excluded from Git. Review Git history for accidentally committed secrets and provide a rotation/remediation plan if any are found.

```
Act as a senior production engineer working on this specific area: 2. GitHub Repository & Source-Control Setup.

TASK
Configure the project as a professional Git repository. Create a clean README, sensible folder structure, branch strategy, .gitignore, environment-variable documentation, issue templates if useful, and CI checks. Ensure secrets, local databases, build artifacts, logs, private keys and service credentials are excluded from Git. Review Git history for accidentally committed secrets and provide a rotation/remediation plan if any are found.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
```

```
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

3. Frontend Deployment — Vercel / Netlify / Cloudflare Pages

Objective: Prepare the frontend for production deployment. Detect whether the framework is Next.js, React/Vite, Vue, Angular or another stack. Configure the correct build command, output directory, Node/runtime version, environment variables and SPA/SSR routing. Connect GitHub so deployments occur through controlled CI/CD. Configure preview deployments and production deployment separately. Verify the deployed frontend can communicate with the production API using HTTPS and correct CORS rules.

```
Act as a senior production engineer working on this specific area: 3. Frontend Deployment — Vercel /
Netlify / Cloudflare Pages.

TASK
Prepare the frontend for production deployment. Detect whether the framework is Next.js, React/Vite, Vue,
Angular or another stack. Configure the correct build command, output directory, Node/runtime version,
environment variables and SPA/SSR routing. Connect GitHub so deployments occur through controlled CI/CD.
Configure preview deployments and production deployment separately. Verify the deployed frontend can
communicate with the production API using HTTPS and correct CORS rules.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

4. Backend Deployment — Railway / Render / Cloud Run / Equivalent

Objective: Prepare the backend/API for production. Detect the framework and runtime. Configure build/start commands, health checks, environment variables, migrations, background jobs and graceful shutdown. Do not use development servers in production. Add structured logging, safe error responses, request IDs and basic monitoring. Restrict CORS to trusted production/preview origins instead of allowing every origin.

```
Act as a senior production engineer working on this specific area: 4. Backend Deployment — Railway /
Render / Cloud Run / Equivalent.

TASK
Prepare the backend/API for production. Detect the framework and runtime. Configure build/start
commands, health checks, environment variables, migrations, background jobs and graceful shutdown. Do
not use development servers in production. Add structured logging, safe error responses, request IDs and
basic monitoring. Restrict CORS to trusted production/preview origins instead of allowing every origin.
```

```
IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

5. Single-Platform Deployment Option

Objective: If the project can be safely deployed on one platform, evaluate an all-in-one architecture. Compare the practical fit of Vercel, Netlify, Render, Cloudflare and similar providers based on the actual framework. Keep the database and secrets in services designed for them. Do not force everything into one provider if doing so creates security, performance, vendor-lock-in or operational problems. Document the final architecture and the reason for choosing it.

```
Act as a senior production engineer working on this specific area: 5. Single-Platform Deployment Option.

TASK
If the project can be safely deployed on one platform, evaluate an all-in-one architecture. Compare the
practical fit of Vercel, Netlify, Render, Cloudflare and similar providers based on the actual framework.
Keep the database and secrets in services designed for them. Do not force everything into one provider if
doing so creates security, performance, vendor-lock-in or operational problems. Document the final
architecture and the reason for choosing it.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

6. Custom Domain, DNS & HTTPS

Objective: Set up a production domain using a registrar such as Cloudflare Registrar, Namecheap, GoDaddy or Hostinger. Configure DNS records correctly, including A/AAAA/CNAME records where required. Enable HTTPS/TLS. Redirect HTTP to HTTPS and choose one canonical hostname, such as www or non-www. Verify DNS propagation, certificate validity, redirect behavior, API subdomains and email-related DNS records. Do not expose unnecessary DNS records.

```
Act as a senior production engineer working on this specific area: 6. Custom Domain, DNS & HTTPS.
```

```
TASK
Set up a production domain using a registrar such as Cloudflare Registrar, Namecheap, GoDaddy or
Hostinger. Configure DNS records correctly, including A/AAAA/CNAME records where required. Enable
HTTPS/TLS. Redirect HTTP to HTTPS and choose one canonical hostname, such as www or non-www. Verify DNS
propagation, certificate validity, redirect behavior, API subdomains and email-related DNS records. Do
not expose unnecessary DNS records.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

7. Cloudflare Layer

Objective: If Cloudflare is used, configure DNS, proxying where appropriate, TLS mode, caching rules, WAF/security rules, rate limiting and bot/abuse protections. Do not cache personalized or authenticated responses. Protect the origin where possible and avoid configuration that breaks WebSockets, APIs or file uploads. Document which records are proxied and why.

```
Act as a senior production engineer working on this specific area: 7. Cloudflare Layer.

TASK
If Cloudflare is used, configure DNS, proxying where appropriate, TLS mode, caching rules, WAF/security
rules, rate limiting and bot/abuse protections. Do not cache personalized or authenticated responses.
Protect the origin where possible and avoid configuration that breaks WebSockets, APIs or file uploads.
Document which records are proxied and why.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

8. Authentication — Email + Google OAuth

Objective: Implement secure authentication using email/password and Google OAuth where appropriate. Use a mature auth provider rather than inventing cryptography. Enforce secure password hashing, email verification where needed, password-reset protection, session expiration/rotation, secure cookies and CSRF protection where applicable.

Validate OAuth redirect URIs and never put OAuth client secrets in frontend code. Test signup, login, logout, reset password, account deletion and unauthorized access.

```
Act as a senior production engineer working on this specific area: 8. Authentication — Email + Google
OAuth.

TASK
Implement secure authentication using email/password and Google OAuth where appropriate. Use a mature
auth provider rather than inventing cryptography. Enforce secure password hashing, email verification
where needed, password-reset protection, session expiration/rotation, secure cookies and CSRF protection
where applicable. Validate OAuth redirect URIs and never put OAuth client secrets in frontend code. Test
signup, login, logout, reset password, account deletion and unauthorized access.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

9. Supabase / Firebase Database, Users & Storage

Objective: Connect the backend to Supabase or Firebase according to the existing stack. Design separate concepts for users, profiles, application data, subscriptions, usage counters, audit events and uploaded files. Apply least-privilege access rules. Never trust user IDs, roles or subscription status supplied by the browser. Enforce authorization server-side/database-side. Add indexes and constraints required for correctness and performance. Configure automated backups and a tested restore process where the provider supports it.

```
Act as a senior production engineer working on this specific area: 9. Supabase / Firebase Database, Users
& Storage.

TASK
Connect the backend to Supabase or Firebase according to the existing stack. Design separate concepts for
users, profiles, application data, subscriptions, usage counters, audit events and uploaded files. Apply
least-privilege access rules. Never trust user IDs, roles or subscription status supplied by the browser.
Enforce authorization server-side/database-side. Add indexes and constraints required for correctness
and performance. Configure automated backups and a tested restore process where the provider supports it.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
```

```
6. Provide a short production verification checklist.
```

10. Storage Quotas & Cleanup

Objective: Implement a safe storage-management system. Track per-user storage usage and quotas. When a quota is reached, clearly inform the user and provide the correct upgrade/extra-storage flow if the business model supports it. Never silently delete valuable user files just because storage is full. If retention/cleanup is required, define a documented policy, warning period, soft-delete/trash period and recovery process. Run cleanup jobs with idempotency and audit logs.

```
Act as a senior production engineer working on this specific area: 10. Storage Quotas & Cleanup.

TASK
Implement a safe storage-management system. Track per-user storage usage and quotas. When a quota is
reached, clearly inform the user and provide the correct upgrade/extra-storage flow if the business model
supports it. Never silently delete valuable user files just because storage is full. If retention/cleanup
is required, define a documented policy, warning period, soft-delete/trash period and recovery process.
Run cleanup jobs with idempotency and audit logs.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

11. Subscription & Usage Limits

Objective: If the product has subscriptions, implement plan limits server-side. Define free, paid and higher-capacity tiers, including storage, API usage, seats, uploads or other measurable resources. The client may display limits, but the server must enforce them. Handle payment success, failure, cancellation, renewal, refunds and webhook retries safely. Do not trust a payment-success value sent from the browser. Make webhook processing idempotent.

```
Act as a senior production engineer working on this specific area: 11. Subscription & Usage Limits.

TASK
If the product has subscriptions, implement plan limits server-side. Define free, paid and
higher-capacity tiers, including storage, API usage, seats, uploads or other measurable resources. The
client may display limits, but the server must enforce them. Handle payment success, failure,
cancellation, renewal, refunds and webhook retries safely. Do not trust a payment-success value sent from
the browser. Make webhook processing idempotent.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
```

```
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

12. Email Infrastructure

Objective: Configure transactional email for verification, password reset, billing events, security alerts and important product notifications. Use a proper email service and verified sending domain. Configure SPF, DKIM and DMARC. Never log full password-reset URLs or sensitive tokens. Add rate limits to password-reset and email-verification requests to prevent abuse.

```
Act as a senior production engineer working on this specific area: 12. Email Infrastructure.

TASK
Configure transactional email for verification, password reset, billing events, security alerts and
important product notifications. Use a proper email service and verified sending domain. Configure SPF,
DKIM and DMARC. Never log full password-reset URLs or sensitive tokens. Add rate limits to password-reset
and email-verification requests to prevent abuse.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

13. Analytics & Product Insights

Objective: Configure Google Analytics or another analytics platform for product measurement. Track useful events such as signup, activation, feature usage, conversion, subscription and errors. Minimize collection of personal and sensitive data. Do not send passwords, authentication tokens, payment data, private document contents or unnecessary personal information to analytics. Add consent/privacy controls where legally required. Define a small event taxonomy rather than tracking everything blindly.

```
Act as a senior production engineer working on this specific area: 13. Analytics & Product Insights.

TASK
Configure Google Analytics or another analytics platform for product measurement. Track useful events
such as signup, activation, feature usage, conversion, subscription and errors. Minimize collection of
personal and sensitive data. Do not send passwords, authentication tokens, payment data, private document
contents or unnecessary personal information to analytics. Add consent/privacy controls where legally
required. Define a small event taxonomy rather than tracking everything blindly.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
```

```
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

14. Error Monitoring, Crash Handling & Uptime

Objective: Add production error monitoring such as Sentry or an equivalent service and configure source maps safely. Capture exceptions, failed requests and performance problems while redacting secrets and sensitive user data. Add health endpoints and uptime monitoring. Create a process for triaging recurring crashes, identifying the most affected user flows and fixing root causes rather than repeatedly suppressing errors.

```
Act as a senior production engineer working on this specific area: 14. Error Monitoring, Crash Handling &
Uptime.

TASK
Add production error monitoring such as Sentry or an equivalent service and configure source maps safely.
Capture exceptions, failed requests and performance problems while redacting secrets and sensitive user
data. Add health endpoints and uptime monitoring. Create a process for triaging recurring crashes,
identifying the most affected user flows and fixing root causes rather than repeatedly suppressing
errors.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

15. Offline Access & Reliability

Objective: If the app benefits from offline use, design a safe offline mode using appropriate browser/mobile storage and service-worker techniques. Clearly separate cached public data from private user data. Never cache authentication secrets in unsafe storage. Define synchronization rules and conflict handling. Test offline startup, reconnection, failed writes, duplicate submissions and stale data.

```
Act as a senior production engineer working on this specific area: 15. Offline Access & Reliability.

TASK
If the app benefits from offline use, design a safe offline mode using appropriate browser/mobile storage
and service-worker techniques. Clearly separate cached public data from private user data. Never cache
authentication secrets in unsafe storage. Define synchronization rules and conflict handling. Test
offline startup, reconnection, failed writes, duplicate submissions and stale data.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
```

```
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

16. User Activity, Feature Usage & Product Analytics

Objective: Implement privacy-conscious usage tracking to understand which features users actually use, where users drop off, common errors and major user problems. Build dashboards for activation, retention, feature adoption, conversion and failures. Do not record sensitive content simply because it is technically possible. Use aggregated or pseudonymous data wherever possible.

```
Act as a senior production engineer working on this specific area: 16. User Activity, Feature Usage &
Product Analytics.

TASK
Implement privacy-conscious usage tracking to understand which features users actually use, where users
drop off, common errors and major user problems. Build dashboards for activation, retention, feature
adoption, conversion and failures. Do not record sensitive content simply because it is technically
possible. Use aggregated or pseudonymous data wherever possible.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

17. SEO & Search Visibility

Objective: Implement technical SEO appropriate to the app. Configure page titles, meta descriptions, Open Graph/Twitter metadata, canonical URLs, sitemap.xml, robots.txt, semantic headings, clean URLs, structured data where applicable and fast loading. Prevent private/authenticated pages and internal dashboards from being indexed. Verify Search Console ownership and monitor indexing errors. Do not create duplicate or spammy pages solely to manipulate rankings.

```
Act as a senior production engineer working on this specific area: 17. SEO & Search Visibility.

TASK
Implement technical SEO appropriate to the app. Configure page titles, meta descriptions, Open
Graph/Twitter metadata, canonical URLs, sitemap.xml, robots.txt, semantic headings, clean URLs,
structured data where applicable and fast loading. Prevent private/authenticated pages and internal
dashboards from being indexed. Verify Search Console ownership and monitor indexing errors. Do not create
duplicate or spammy pages solely to manipulate rankings.
```

```
IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

18. Trademark, Brand & Domain Review

Objective: Before public launch, check whether the chosen product name, logo and domain create obvious trademark or brand conflicts. Search relevant trademark databases for the target countries and document potential conflicts. Do not assume that an available domain means the brand is legally safe. If the brand is important to the business, obtain professional legal advice before investing heavily in it.

```
Act as a senior production engineer working on this specific area: 18. Trademark, Brand & Domain Review.

TASK
Before public launch, check whether the chosen product name, logo and domain create obvious trademark or
brand conflicts. Search relevant trademark databases for the target countries and document potential
conflicts. Do not assume that an available domain means the brand is legally safe. If the brand is
important to the business, obtain professional legal advice before investing heavily in it.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

19. Code Review & Automated Security Review

Objective: Run a full code review using tools such as CodeRabbit, CodeAnt, Codex-assisted review, the framework's lint/type checks, dependency scanners and secret scanners. Treat AI/tool findings as leads, not unquestionable truth. Manually verify high-risk findings. Check authentication, authorization, injection risks, insecure direct object references, SSRF, XSS, CSRF, unsafe file uploads, dependency vulnerabilities, exposed secrets, weak CORS, insecure redirects and error leakage. Fix root causes and rerun tests.

```
Act as a senior production engineer working on this specific area: 19. Code Review & Automated Security
Review.
```

```
TASK
Run a full code review using tools such as CodeRabbit, CodeAnt, Codex-assisted review, the framework's
lint/type checks, dependency scanners and secret scanners. Treat AI/tool findings as leads, not
unquestionable truth. Manually verify high-risk findings. Check authentication, authorization, injection
risks, insecure direct object references, SSRF, XSS, CSRF, unsafe file uploads, dependency
vulnerabilities, exposed secrets, weak CORS, insecure redirects and error leakage. Fix root causes and
rerun tests.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

20. API & Third-Party Integration Review

Objective: Inventory every external API and integration. Document credentials, scopes, rate limits, webhooks, failure modes and data shared with each provider. Keep secrets server-side. Add timeouts, retries with backoff, circuit-breaking where appropriate and safe error handling. Validate webhook signatures and make webhook handlers idempotent. Never expose privileged third-party API keys in frontend bundles.

```
Act as a senior production engineer working on this specific area: 20. API & Third-Party Integration
Review.

TASK
Inventory every external API and integration. Document credentials, scopes, rate limits, webhooks,
failure modes and data shared with each provider. Keep secrets server-side. Add timeouts, retries with
backoff, circuit-breaking where appropriate and safe error handling. Validate webhook signatures and
make webhook handlers idempotent. Never expose privileged third-party API keys in frontend bundles.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

21. India + International Documents, Tax & Compliance Readiness

Objective: If the product collects or processes tax numbers, passport information, identity documents, invoices or other regulated/personal information, design for data minimization, encryption, access control, retention and deletion.

Determine which countries the service will operate in and identify relevant privacy, tax, consumer-protection and identity-document requirements. For India, review applicable GST/tax and privacy requirements; for other countries, map requirements based on the actual markets. Do not present the implementation as legal advice; create a compliance checklist for qualified professionals to validate.

```
Act as a senior production engineer working on this specific area: 21. India + International Documents, Tax & Compliance Readiness.

TASK
If the product collects or processes tax numbers, passport information, identity documents, invoices or other regulated/personal information, design for data minimization, encryption, access control, retention and deletion. Determine which countries the service will operate in and identify relevant privacy, tax, consumer-protection and identity-document requirements. For India, review applicable GST/tax and privacy requirements; for other countries, map requirements based on the actual markets. Do not present the implementation as legal advice; create a compliance checklist for qualified professionals to validate.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

22. Web-to-Mobile / Play Store Conversion

Objective: If converting the web product into a mobile app, first determine whether a responsive/PWA approach is sufficient or whether a native/cross-platform app is justified. Reuse the backend APIs and authentication safely. Add mobile-specific secure storage, deep links, push notifications and offline handling only when needed. For Android/Google Play, prepare package ID, signing keys, app icon, screenshots, privacy policy, data-safety disclosures and store listing. Never commit signing keys or service-account credentials to Git.

```
Act as a senior production engineer working on this specific area: 22. Web-to-Mobile / Play Store Conversion.

TASK
If converting the web product into a mobile app, first determine whether a responsive/PWA approach is sufficient or whether a native/cross-platform app is justified. Reuse the backend APIs and authentication safely. Add mobile-specific secure storage, deep links, push notifications and offline handling only when needed. For Android/Google Play, prepare package ID, signing keys, app icon, screenshots, privacy policy, data-safety disclosures and store listing. Never commit signing keys or service-account credentials to Git.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the provider-specific setup rather than pretending it is configured.

DELIVERABLES
```

```
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

23. Final Production QA, Launch Checklist & Continuous Maintenance

Objective: Before launch, perform a complete production-readiness test. Verify signup/login, OAuth, password reset, authorization, database writes, uploads, storage quotas, subscriptions, payments/webhooks if present, email delivery, analytics, error monitoring, SEO, DNS, HTTPS, mobile responsiveness, accessibility, performance and critical user journeys. Test backups and restore. Verify rollback. Create a maintenance schedule for dependency updates, security reviews, backups, domain certificates, DNS, monitoring alerts, storage cleanup, analytics review and user-feedback review. Produce a launch report with PASS/FAIL status and remaining risks.

```
Act as a senior production engineer working on this specific area: 23. Final Production QA, Launch
Checklist & Continuous Maintenance.

TASK
Before launch, perform a complete production-readiness test. Verify signup/login, OAuth, password reset,
authorization, database writes, uploads, storage quotas, subscriptions, payments/webhooks if present,
email delivery, analytics, error monitoring, SEO, DNS, HTTPS, mobile responsiveness, accessibility,
performance and critical user journeys. Test backups and restore. Verify rollback. Create a maintenance
schedule for dependency updates, security reviews, backups, domain certificates, DNS, monitoring alerts,
storage cleanup, analytics review and user-feedback review. Produce a launch report with PASS/FAIL status
and remaining risks.

IMPLEMENTATION REQUIREMENTS
- Inspect the current codebase before changing anything.
- Reuse the existing architecture when it is sound.
- Make production-safe changes rather than demo-only changes.
- Keep secrets out of source code and client bundles.
- Add validation, authorization and useful error handling.
- Add or update tests for important behavior.
- Document required environment variables and provider settings.
- Do not remove existing functionality unless it is unsafe or clearly obsolete.
- If a requirement depends on a paid provider, implement the integration cleanly and document the
provider-specific setup rather than pretending it is configured.

DELIVERABLES
1. List the files/components that need changes.
2. Implement the changes.
3. Add tests and run them.
4. Report any configuration that must be completed in the provider dashboard.
5. Report remaining risks or assumptions.
6. Provide a short production verification checklist.
```

24. One-Shot Master Prompt for a Vibe-Coding / AI Development Tool

Use this when you want an AI coding agent to audit and complete the entire app rather than handling each section separately.

```
You are responsible for taking my existing web app from its current state to production-ready.

Do NOT start by rewriting the application. First inspect everything.

A. DISCOVER
- Identify framework, language, package manager, build system and runtime.
- Identify frontend, backend/API, database, storage, authentication, payments, email, analytics and
external APIs.
- Identify all routes, server actions, API endpoints and privileged operations.
- Identify current hosting/domain/DNS configuration.
- Identify all environment variables and whether any secret is exposed client-side.
- Identify existing tests and CI/CD.

B. ARCHITECTURE
Choose the simplest reliable production architecture. Typical options:
- Frontend: Vercel / Netlify / Cloudflare Pages.
- Backend: Railway / Render / Cloud Run / equivalent.
- Database/auth/storage: Supabase or Firebase when appropriate.
- DNS/CDN/security: Cloudflare when appropriate.
- Source control: GitHub.
- Monitoring: Sentry + provider logs + uptime monitoring.
Do not force a provider choice when the current stack is better served elsewhere.

C. SECURITY
Audit authentication, authorization, CORS, CSRF, XSS, SQL/NoSQL injection, SSRF, IDOR, file uploads,
rate limits, brute force, replay attacks, secret handling, dependency vulnerabilities, error leakage,
insecure redirects, webhook verification and privilege escalation.
Apply least privilege and server-side enforcement.

D. DEPLOYMENT
Connect GitHub to the selected hosting platform. Configure preview and production environments.
Configure environment variables through the platform, not Git. Build and deploy the frontend and backend.
Configure database migrations and health checks. Verify production URLs.

E. DOMAIN
Connect the purchased domain. Configure DNS, TLS, canonical hostname and HTTPS redirects. Verify all DNS
records required for email and third-party services.

F. AUTH + USERS
Implement secure email authentication and Google OAuth if required. Add verification, password reset,
session handling and account deletion. Store only the minimum required personal information.

G. DATABASE + STORAGE
Create safe schema, indexes, constraints and access rules. Configure backups. Track storage usage and
plan limits. Do not silently delete user data when storage is full. Use warning/upgrade/retention
workflows.

H. SUBSCRIPTIONS
If monetized, enforce plan limits server-side and process payment webhooks idempotently. Never trust the
frontend for payment state.

I. EMAIL + ANALYTICS
Configure transactional email with SPF/DKIM/DMARC. Configure privacy-conscious analytics and product
events. Never send secrets or private document contents to analytics.

J. MONITORING
Configure crash/error monitoring, health checks, uptime monitoring and alerting. Redact tokens,
passwords and sensitive data from logs.

K. SEO
Configure metadata, sitemap, robots, canonical URLs, structured data where appropriate, performance and
indexability. Keep private pages out of search indexes.

L. BRAND/COMPLIANCE
Review domain/trademark conflicts and create a checklist for applicable privacy, tax, consumer and
```

```
identity-document requirements in India and other target countries. Do not provide legal conclusions
without qualified review.

M. CODE REVIEW
Run lint, type checks, tests, dependency scanning, secret scanning and AI-assisted code review. Manually
verify high-risk findings.

N. MOBILE
If mobile is required, evaluate PWA vs native/cross-platform. Prepare Android/Play Store requirements
without exposing signing keys.

O. QA + LAUNCH
Test all critical user journeys on production. Verify backups/restore, rollback, monitoring, DNS, HTTPS,
email, auth, subscriptions and data integrity.

FINAL OUTPUT
Return:
1. Architecture diagram in text.
2. Files changed.
3. Environment variables required.
4. Provider dashboard settings required.
5. Database migrations.
6. Security findings and fixes.
7. Tests run and results.
8. Deployment instructions.
9. Launch checklist.
10. Maintenance checklist.
11. Remaining risks.

Never claim something is configured if you did not actually configure or verify it.
```

25. Recommended Maintenance Schedule

Daily / automated	Monitor uptime, error alerts, failed jobs, suspicious auth activity, payment/webhook failures and storage failures.
Weekly	Review top errors, feature usage, user complaints, failed signups, performance regressions and unusual traffic.
Monthly	Update dependencies, review access permissions, check storage growth, review analytics, verify domain/email health and inspect security alerts.
Quarterly	Perform a security review, backup-restore test, access-control audit, SEO review, cost review and disaster-recovery exercise.
Before every major release	Run full regression tests, dependency/security scan, database migration test, rollback test and production smoke test.

Important: The prompts are intentionally provider-aware but not provider-blind. The AI/developer should inspect the actual stack first and choose the simplest secure architecture rather than mechanically installing every listed service.